future internet

MDPI

Article

Towards a Protocol-Aware Intrusion Detection System for LoRaWAN Networks

Zsolt Bringye [1], Rita Fleiner [1,*] and Eszter Kail [1,2]

[1] John von Neumann Faculty of Informatics, Obuda University, 1034 Budapest, Hungary; bringye.zsolt@nik.uni-obuda.hu (Z.B.); kail.eszter@nik.uni-obuda.hu or kail.eszter@sztaki.hun-ren.hu (E.K.)
[2] Institute for Computer Science and Control (HUN-REN SZTAKI), HUN-REN Hungarian Research Network, 1111 Budapest, Hungary
* Correspondence: fleiner.rita@nik.uni-obuda.hu

Abstract

The increasing reliance of Internet of Things (IoT) applications on low-power wide-area network technologies, particularly Long Range Wide Area Network (LoRaWAN), has amplified the need for security monitoring approaches that go beyond attack-specific signatures and generic traffic anomalies. Existing solutions are often tailored to individual threat scenarios or rely on statistical indicators, which limits their ability to systematically capture protocol-level misuse in an interpretable manner. This paper addresses this gap by proposing a protocol-aware validation methodology based on a Digital Twin abstraction of LoRaWAN communication behavior. The Over-The-Air Activation (OTAA) procedure is modeled as a finite-state machine that encodes expected message sequences, timing constraints, and specification-driven state transitions. Observed network events are continuously evaluated against this formal state model, enabling the identification of protocol-level deviations indicative of anomalous or non-conformant behavior. Illustrative examples include replay behavior, timing inconsistencies, and integrity-related anomalies, although the framework is not limited to predefined attack categories. The results demonstrate that state machine-based Digital Twin provides a structured and extensible foundation for protocol-aware security validation and Security Operation Center (SOC)-oriented telemetry enrichment. In this sense, the presented approach represents a concrete step toward protocol-aware intrusion detection for LoRaWAN networks by establishing a state-synchronized semantic validation layer upon which higher-level detection mechanisms can be built.

Keywords: LoRaWAN security; anomaly detection; SIEM; OTAA; IDS; digital twin; FSM

Academic Editor: Cheng-Chi Lee

Received: 25 January 2026
Revised: 4 March 2026
Accepted: 6 March 2026
Published: 9 March 2026

1. Introduction

Internet of Things (IoT) systems operate under severe technical and environmental constraints, including limited bandwidth, restricted computational capabilities, energy-efficiency requirements, and exposure to wireless interference. These constraints significantly complicate anomaly and intrusion detection, as security mechanisms must function under sparse traffic patterns, partial observability, and strict operational budgets.

In wireless sensor and IoT environments, measurement uncertainty, environmental noise, communication unreliability, and distributed coordination overhead further increase system-level complexity, often requiring resource-aware distributed processing and lightweight validation mechanisms to maintain operational reliability. Recent studies

emphasize that real-world IoT deployments therefore require protocol-aware and resource-conscious security and data validation mechanisms capable of operating reliably within these limitations [1,2].

These systemic constraints become particularly pronounced in low-power wide-area networks, where communication sparsity, strict duty-cycle limitations, and encrypted payload structures further restrict observability and security monitoring.

Long Range Wide Area Network (LoRaWAN) exemplifies these challenges. Although the protocol provides cryptographic protection and energy-efficient long-range communication, its sparse and largely encrypted traffic, distributed gateway architecture, and strongly state-dependent activation process restrict the applicability of conventional intrusion detection mechanisms. Security-relevant deviations during the Over-The-Air Activation (OTAA) procedures often manifest as protocol-level inconsistencies rather than malformed packets or volumetric anomalies.

Existing LoRaWAN intrusion detection approaches typically rely on statistical analysis of join behavior, DevNonce patterns, or traffic-level indicators. While effective against certain attack classes, such methods generally do not explicitly model protocol state evolution. Consequently, syntactically valid yet semantically inconsistent protocol executions may remain undetected.

This motivates the following research problem: how can protocol-level semantic correctness be continuously validated in LoRaWAN networks under constrained observability and encrypted communication?

Addressing this challenge requires moving beyond purely data-driven and heuristic detection mechanisms. Prior work in cyber-physical and industrial systems has demonstrated the effectiveness of model-driven and specification-based techniques for security monitoring. In such settings, formal behavioral models enable systematic reasoning about correctness and the detection of deviations from expected operation without relying on learning-based methods [3,4]. These results suggest that explicit protocol and state modeling can provide a strong foundation for interpretable and verifiable intrusion detection in distributed, resource-constrained environments.

This paper introduces a protocol-aware, state-synchronized validation layer designed for integration into Security Operation Center (SOC)-oriented telemetry pipelines. The proposed approach formalizes the OTAA procedure as an executable finite-state model and continuously validates observed communication against specification-level state progression. The resulting structured, state-aware outputs enrich Security Information and Event Management (SIEM) analytics and provide a foundation for protocol-conformant security monitoring in LoRaWAN deployments.

The main contributions of this work are:

- A finite state machine-based formalization of the LoRaWAN OTAA procedure at the specification level.
- An executable protocol-aware validation layer capable of detecting semantically inconsistent yet syntactically valid protocol behavior.
- Integration of the state-aware model into a SOC-compatible preprocessing pipeline.
- A foundation for extending LoRaWAN monitoring toward fully state-synchronized intrusion detection architectures.

The remainder of this paper is organized as follows. Section 2 provides background on LoRaWAN communication, with particular emphasis on the OTAA procedure and its protocol-level observability. Section 3 reviews related work, covering known security weaknesses of OTAA procedures as well as existing intrusion detection and digital twin-based approaches in IoT networks, and positions the present work within this landscape. Section 4 introduces the proposed methodology, including the threat model, observability

assumptions, and protocol-aware Digital Twin realized as a finite-state machine. This section details the hierarchical rule system, formal semantics, and state transition logic used for protocol-conformance validation. Section 5 describes the experimental setup and system implementation, including the LoRaWAN testbed, monitoring points, preprocessing pipeline, and SIEM integration. Section 6 presents the evaluation results obtained from controlled OTAA scenarios, illustrating Finite State Machine (FSM) state evolution, detection outcomes, and SOC-level interpretability. Finally, Section 7 discusses the implications, limitations, and future extensions of the proposed approach and Section 8 concludes this paper.

2. Background

2.1. LoRa, LoRaWAN

LoRaWAN [5] is a low-power wide-area networking protocol designed to support long-range, energy-efficient communication for large-scale IoT deployments. It operates on top of LoRa radio technology [6], which employs chirp spread spectrum modulation to achieve robust communication under low signal-to-noise conditions while enabling multi-year battery lifetimes for resource-constrained end devices.

Beyond the physical layer, LoRaWAN defines both the communication protocol and the system architecture governing device-to-cloud interaction. The network follows a star-of-stars topology in which end devices transmit LoRa frames directly to one or more gateways. Gateways act as transparent forwarders, relaying received radio packets to backend servers over IP-based networks without maintaining the LoRaWAN protocol state. This design enables redundancy, multi-gateway reception, and scalable network operation.

The backend infrastructure is logically decomposed into functional components with clearly separated responsibilities. The Network Server performs MAC-layer processing, including frame validation, message deduplication, adaptive data rate control, downlink scheduling, and device state tracking. Application Servers process application-layer payloads and expose data to end-user services. Newer LoRaWAN specifications [7] formalize the Join Server as a dedicated component responsible for authentication and join-related key management, reinforcing security separation within the architecture.

LoRaWAN defines multiple operational modes for end devices to balance energy efficiency and downlink availability. Class A operation, which is mandatory for all devices, provides the lowest energy consumption by opening short receive windows only after uplink transmissions. Class B and Class C introduce progressively more predictable downlink reception at the cost of increased energy usage. These class-dependent behaviors result in distinct communication patterns and timing characteristics that are observable at the network level.

Communication in LoRaWAN relies on a small set of well-defined message types, including join messages and data frames. Data frames follow a layered structure consisting of a MAC header, a MAC payload, and a Message Integrity Code (MIC). The MAC payload contains addressing and control information, frame counters, optional MAC commands, and the encrypted application payload. This structure enables both application data transfer and protocol control signaling within a single cryptographically protected frame. At the physical layer, the LoRaWAN payload is encapsulated within a LoRa PHY frame, whose size and format depend on regional parameters and data rate settings. Security is a core design principle of LoRaWAN. The aforementioned message structure is depicted in Figure 1.

The protocol employs symmetric cryptography and session-based keys to ensure confidentiality and integrity of communication. Message integrity is provided by the MIC appended to each frame, allowing detection of unauthorized modifications. The exact

fields included in the MIC computation depend on the message type, implicitly binding integrity verification to the protocol state.

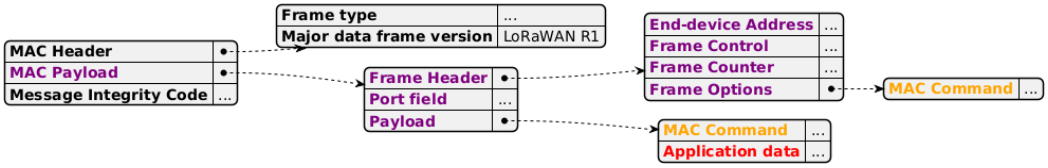

Figure 1. Structure of Data Frame [8].

Device activation is performed either through OTAA procedure or Activation by Personalization. OTAA procedure dynamically establishes a session and derives fresh session keys, while Activation by Personalization relies on statically provisioned parameters and does not involve a runtime join exchange. Because join procedures and key derivation differ across specification versions, and to remain consistent with both our experimental environment and prevailing deployments, this work adopts the LoRaWAN 1.0.3 specification. Although LoRaWAN 1.1 introduces notable security improvements, version 1.0.3 remains widely deployed due to its maturity, compatibility with legacy devices, and continued support in commonly used network server implementations such as ChirpStack. Basing the analysis on LoRaWAN 1.0.3 therefore allows the presented results to reflect realistic operational conditions.

In addition to the transmitted payload, LoRaWAN communication is enriched with contextual metadata during reception and forwarding. Gateways typically attach radio-level attributes such as received signal strength indicator (RSSI), signal-to-noise ratio (SNR), frequency, channel, and timestamps to each received frame and periodically report their own operational status. Network servers further augment messages with device registry information and session state. When join context and cryptographic keys are available, application payloads can be decrypted, enabling deeper inspection. This multi-layer enrichment across radio, MAC, and application layers provides a rich observation space and forms the foundation for the state-aware modeling and anomaly detection approach presented in the remainder of this paper.

2.2. OTAA

Over-the-Air Activation (OTAA) is the preferred activation mechanism in LoRaWAN, enabling dynamic session establishment and per-session key derivation. The join procedure defines the initial protocol state of an end device and establishes the cryptographic context for subsequent communication, making it a security-critical phase of the protocol.

The OTAA process is initiated by the end device through a `Join Request` uplink message. This message contains the device and application identifiers (DevEUI, JoinEUI/AppEUI) and a device-generated DevNonce, which is intended to be unique for each join attempt. The `Join Request` payload is not encrypted but is protected by a Message Integrity Code (MIC) computed using the root application key (AppKey), ensuring authenticity and integrity. The presence, reuse, or timing of DevNonce values is therefore a key observable during activation. Upon successful validation, the network responds with a `Join Accept` downlink message, which is encrypted and authenticated using the AppKey. The `Join Accept` assigns a dynamic DevAddr and provides essential network parameters, including AppNonce, NetID, DLSettings, and RXDelay, with an optional CFList depending on regional configuration. Using the exchanged nonces and the AppKey, the device derives session keys for MAC-layer integrity and application payload encryption.

In LoRaWAN version 1.0.3, rejoining is typically realized by repeating the standard OTAA procedure rather than using explicit rejoin message types. Rejoin events may occur

after device reboot, session expiration, or prolonged communication failures. While such behavior can be benign, repeated or irregular join attempts, inconsistent timing, or nonce reuse may also indicate misconfiguration or malicious interference.

3. Related Work

3.1. Security Weaknesses and Attack Surfaces of OTAA

The OTAA procedure plays a central role in establishing secure LoRaWAN communication; however, while intended to offer superior security over ABP, the design of LoRaWAN version 1.0.x introduces several security-relevant weaknesses and vulnerabilities [9,10]. These weaknesses primarily stem from the limited entropy of nonces, the lack of cryptographic binding between join messages, and the centralized trust model where the network server manages all encryption keys [11]. Many of these issues do not arise from malformed packets or broken cryptography but from violations of expected protocol sequencing, timing, or value reuse. As a result, they are particularly difficult to detect using stateless or signature-based approaches.

3.1.1. DevNonce Reuse and Replay Conditions

In LoRaWAN 1.0.3, the DevNonce used in `Join Requests` messages is a 16-bit value generated by the end device and intended to be unique per join attempt [12]. This design is inherently susceptible to the birthday paradox, where the probability of generating a duplicate nonce increases rapidly over the device's lifetime, estimated at a 10-year operational span [11]. In practice, constrained devices may reuse DevNonce values due to reboots, counter resets, or poor randomness. While the specification mandates DevNonce uniqueness, enforcement relies on network-side tracking of previously observed values.

Furthermore, many low-cost IoT devices, such as those utilizing the SX1272 transceiver, rely on the least significant bits (LSBs) of RSSI measurements as an entropy source for random number generation [13]. Experimental data indicate that such hardware-based generators often lack health tests and exhibit predictable patterns, which can be further compromised through targeted RF jamming [11]. DevNonce reuse enables replay-based attack scenarios, where previously captured `Join Requests` are retransmitted to trigger unintended join behavior or desynchronize session states. Importantly, such messages are syntactically valid and protected by a correct MIC, making them indistinguishable from legitimate traffic when evaluated in isolation. Detection therefore requires stateful tracking of DevNonce values across join attempts and correlation with prior protocol history.

3.1.2. Join Request Flooding and Activation Abuse

The OTAA procedure is uplink-driven and may be repeatedly triggered by an end device in the absence of a successful `Join Accept`. This behavior is expected under benign conditions such as downlink loss or temporary network unavailability. However, excessive or patterned `Join Requests` transmissions can also be exploited to cause resource exhaustion at the network server, increase gateway load, or disrupt legitimate device activation.

Also, in LoRaWAN version 1.0.x, the `Join Requests` message is transmitted in plaintext, which exposes the device's identifiers (DevEUI and AppEUI/JoinEUI) to passive observers. This lack of confidentiality, combined with the protocol's retry mechanisms, enables several abuse scenarios. An adversary can perform a self-replay attack by selectively jamming the Network Server's `Join Accept` response. Following the specification, the end device will retransmit the original `Join Requests` using the same DevNonce, allowing the attacker to force the device into repeating this cycle until its daily message quota (e.g., 14 packets per day) is depleted, effectively rendering the device unreachable [13]. Such abuse does not necessarily violate packet-level constraints. Instead, it manifests as abnor-

mal activation frequency, inconsistent timing, or prolonged residence in the joining state. Distinguishing benign retries from malicious behavior requires reasoning about expected retry behavior, timing constraints, and device state evolution.

3.1.3. Join Accept Manipulation and Desynchronization Effects

A fundamental flaw in LoRaWAN version 1.0.x is that the `Join Accept` message is not cryptographically bound to the preceding Join Request. The payload and Message Integrity Code (MIC) of the `Join Accept` are independent of the DevNonce provided by the device. This vulnerability allows an adversary to perform a `Join Accept` replay attack, where a previously captured valid `Join Accept` is retransmitted in response to a new `Join Request` [11].

Loss or manipulation of `Join Accept` messages, whether due to radio conditions or adversarial interference, can lead to state desynchronization [11], where the network considers a device joined while the device remains unactivated. Such desynchronization may cause repeated join attempts, inconsistent session key usage, or unexpected message sequences. From a monitoring perspective, these situations appear as valid messages occurring in unexpected protocol states, rather than explicit errors.

3.1.4. Timing and State Transition Violations

The OTAA procedure imposes implicit timing relationships between uplink `Join Requests` and downlink `Join Accept` reception windows. Deviations from expected timing, such as texttttJoin Accept messages observed outside valid receive windows or join-related messages appearing after an established session, represent violations of protocol progression rather than structural anomalies. Formal verification using Timed Automata has shown that even minor timing drifts can lead to the loss of downlink synchronization [10]. These timing-based inconsistencies are particularly relevant in environments with multiple gateways and asynchronous forwarding, where reordered or duplicated observations may occur. Correct interpretation therefore requires explicit modeling of allowed state transitions and their temporal constraints.

3.2. Intrusion Detection Systems and Digital Twin Solutions in IoT Networks

Intrusion Detection Systems (IDSs) in IoT environments have evolved into a broad spectrum of architectural and methodological approaches, reflecting the diversity and complexity of the systems they aim to protect. Depending on latency, privacy, and computational constraints, IDS solutions can be deployed across edge, fog, and cloud layers [14,15].

Beyond architectural placement, IDS solutions are commonly classified according to their detection logic. In addition to signature- and anomaly-based techniques, stateful protocol analysis (often referred to as specification-based intrusion detection) relies on formal protocol specifications and explicit state tracking to identify deviations from expected behavior [16]. This class of IDSs is particularly well suited to protocol-driven and wireless communication systems, where security-relevant anomalies often manifest as violations of message ordering, timing constraints, or state-dependent invariants rather than isolated packet-level features.

However, the growing protocol diversity and stateful nature of IoT communications pose significant challenges for conventional IDS designs, particularly in low-power and delay-sensitive networks, where scalability and protocol specificity are critical.

To address these limitations, several studies have explored structured, protocol-aware detection mechanisms that incorporate formal models such as FSMs. Beyond purely statistical detection, explicit state modeling enables IDS solutions to reason about protocol semantics rather than aggregate traffic behavior. By encoding valid message sequences, timing constraints, and state-dependent invariants, FSM-based approaches can detect

subtle violations that remain indistinguishable at the packet or feature level. This capability is particularly important in protocols such as LoRaWAN, where many security-relevant deviations manifest only across multiple messages or reception windows.

In parallel, Digital Twin paradigms have gained attention as a means to maintain a virtual representation of device behavior or protocol execution, enabling the detection of deviations between expected and observed states. While the notion of a digital twin varies across application domains, a common characteristic is the continuous synchronization between the physical system and its virtual counterpart. In many IoT security applications, digital twins primarily serve as contextual mirrors or simulation environments. In contrast, protocol-aware digital twins emphasize real-time synchronization and enforcement, allowing deviations from expected protocol behavior to be detected as part of normal system operation. These approaches are especially relevant for lightweight IoT technologies such as LoRaWAN, where payload inspection is limited, but protocol-level inconsistencies, such as unauthorized joins, replay attempts, or timing violations can be effectively identified through state-driven logic.

The challenge of detecting protocol-level misuse is not unique to LoRaWAN and has been observed in other wireless communication systems as well. Recent research has demonstrated that formally valid Bluetooth Low Energy (BLE) signaling can be abused for covert data exfiltration without violating low-level constraints. By hijacking Offline Finding Networks (OFNs), attackers can encode data within protocol-compliant announcement keys, rendering the activity invisible to traditional radio- or signature-based detection mechanisms [17].

This cross-domain example illustrates that protocol-compliant signaling may still enable semantically malicious behavior. While the BLE case involves a different protocol ecosystem, it highlights the broader need for semantic, state-aware validation beyond radio-level checks and motivates protocol-aware enforcement mechanisms capable of identifying syntactically valid yet semantically malicious protocol behavior.

3.2.1. State-Based IDS Approaches for LoRaWAN

The stateful nature of the LoRaWAN OTAA procedure has motivated several intrusion detection approaches based on protocol modeling. Danish et al. [18] propose LIDS, an IDS that monitors the OTAA join process to detect jamming attacks. Their method analyzes DevNonce values using statistical divergence metrics, such as Kullback–Leibler divergence and Hamming distance, achieving high detection accuracy. While their approach implicitly captures protocol state through statistical modeling, it does not explicitly represent the join procedure as a formal state machine, which limits interpretability and protocol coverage.

Similarly, Horák et al. [19] investigate denial-of-service attacks targeting the OTAA procedure by analyzing `Join-Request` patterns at the gateway level. Their method relies on DevNonce randomness analysis and static statistical indicators derived from gateway logs. Although state-aware in spirit, their solution does not model the full protocol state evolution and remains limited to partial observability at a single monitoring point. These studies illustrate that while statistical methods can detect certain anomalies, they struggle to distinguish between benign retransmissions, gateway artifacts, and protocol-level violations without explicit state modeling.

Beyond LoRaWAN-specific solutions, several studies demonstrate the effectiveness of specification-based and automata-driven IDSs in constrained IoT environments. Fu et al. [20] introduce an automata-based detection framework for heterogeneous IoT protocols, using labeled transition systems to identify replay, jamming, and message forgery attacks. Likewise, Le et al. [21] propose an FSM-based IDS for Routing Protocol for Low Power Lossy Networks, where deviations from expected protocol state transitions are used

to detect routing-level attacks. Although these works target protocols other than LoRaWAN, they underline the value of explicit protocol state modeling for detecting logic-level attacks in low-power and lossy networks.

3.2.2. Digital Twin-Based Anomaly Detection in IoT

Fuller et al. [22] provide a comprehensive conceptual overview of digital twin technologies, explicitly distinguishing between digital models, digital shadows, and fully integrated digital twins based on the directionality and continuity of data exchange between the physical and virtual systems. This distinction is particularly relevant for protocol-centric IoT systems, where the digital twin often acts as a behavioral and state-based reference model rather than a physics-based replica. In such settings, the primary value of a digital twin lies in its ability to maintain synchronized protocol state and expected behavior, which can be leveraged for monitoring and anomaly detection.

Eckhart and Ekelhart in [23] propose a security-aware digital twin framework for cyber-physical systems in which the virtual replica is automatically derived from formal system specifications and continuously synchronized with the physical system. In their approach, the digital twin serves as a reference model for monitoring and intrusion detection, enabling the identification of deviations from expected behavior without relying on learning-based techniques. This work demonstrates that specification-driven, state-aware detection mechanisms can provide effective protection; however, the proposed framework remains conceptual and is not integrated into an operational IDS or SOC-oriented detection pipeline.

Homaei et al. [24] present a multi-layered security architecture for smart water infrastructures, combining a LoRaWAN-based sensor network with machine learning-driven IDS components and a digital twin. In their system, the digital twin primarily serves as a trusted representation of infrastructure behavior and a decision-support tool, while intrusion detection is performed by statistical and learning-based models. As such, the digital twin remains auxiliary to detection rather than constituting the detection logic itself.

El-Hajj et al. [25] integrate a real-time digital twin with a Snort-based IDS in smart city environments, using the twin to mirror device-level metrics such as CPU usage and connectivity state. While effective against volumetric network attacks, the digital twin in their framework does not encode protocol semantics and remains decoupled from intrusion detection decisions.

Schösser et al. [26] employ a digital twin of the radio environment to detect jamming attacks by comparing predicted and observed signal strength values. Their approach demonstrates strong performance at the physical layer but relies on a static twin model and does not incorporate a network or protocol-level state.

In contrast to prior approaches that use digital twins primarily for monitoring or simulation, the present work adopts a protocol-level digital twin concept aimed at validating LoRaWAN communication semantics and enriching SOC-level analytics with state-aware context.

4. Methodology and Prior Work

4.1. Threat Model and Observability Assumptions

The proposed framework models LoRaWAN communication as an FSM-based Digital Twin representing the formal progression of the OTAA protocol. The Digital Twin evaluates observed events against deterministic state transitions and guard conditions derived from the protocol specification [5]. Protocol-level inconsistencies are identified as violations of formally defined state semantics, independent of adversary capability assumptions.

In backend-integrated LoRaWAN deployments, protocol metadata and cryptographic session context are inherently available as part of normal network-server operation. In this configuration, the Digital Twin can decode, authenticate, and formally validate all relevant protocol fields, including integrity-protected elements.

At the same time, protocol-conformance validation does not depend exclusively on cryptographic access. Even without session context, a substantial portion of security-relevant semantics remains observable through protocol metadata and gateway reception attributes. Although metadata-only observability does not enable validation of encrypted `Join Accept` contents or all protected fields, it supports reliable detection of protocol-level inconsistencies that manifest in observable behavior, including abnormal timing patterns, invalid reception-window alignment, inconsistent transmission parameters (e.g., frequency or data rate deviations), and state-progression violations at the monitoring point.

The security relevance of both metadata and decoded protocol fields is inherently state-dependent: identical values or message structures may indicate benign or malicious behavior depending on the current phase of the protocol. This observation motivates state-aware detection mechanisms that explicitly reason about protocol progression rather than isolated packets.

4.2. Prior Work: SOC-Compatible Telemetry Pipeline

In our earlier work, we addressed the problem of observability by designing a SOC-compatible telemetry pipeline for LoRaWAN networks, without enforcing explicit protocol semantics [8].

While this prior framework demonstrated that security-relevant inconsistencies can be surfaced from protocol metadata alone, detection remained largely correlation-based. In particular, protocol violations spanning multiple messages or timing windows could not be robustly identified without an explicit representation of protocol state.

4.3. Approach Overview: Protocol-Aware Digital Twin

Building on the telemetry pipeline, the Digital Twin functions as a protocol-aware validation layer maintaining a LoRaWAN-aligned communication context. By tracking state variables and cryptographic parameters across events, the FSM enables continuous protocol-conformance validation within the monitoring workflow.

The Digital Twin continuously performs protocol semantic checks. Incoming events are evaluated against the expected protocol progression, allowing the system to detect violations such as invalid message ordering, nonce reuse, or timing inconsistencies that cannot be reliably identified through stateless correlation alone.

This design enables intrusion detection to extend beyond packet-level anomaly spotting toward protocol-level conformance checking, thereby supporting improved detection precision and interpretability.

4.4. FSM-Based Protocol-Aware Validation

To strengthen the security and correctness of the LoRaWAN OTAA procedure, we model protocol execution using a rule-based FSM that captures both valid protocol progression and security-relevant failure modes. Beyond the nominal join workflow, the model explicitly encodes constraints related to malformed messages, protocol misuse, replay attempts, and timing violations commonly exploited in real-world LoRaWAN attacks.

The FSM is implemented using a domain-specific language (DSL) embedded in Python [27], allowing protocol behavior to be expressed declaratively as guarded state transitions. Each observed event, either a message reception or a timer expiration, is evaluated against a structured set of protocol invariants covering radio-layer compliance, state-dependent message ordering, cryptographic integrity, replay protection, and reception

window timing. Events violating these constraints are explicitly annotated with structured, state-aware metadata for downstream SIEM integration.

Operationally, protocol rules are validated in a layered hierarchy (Level 0—physical and radio-layer compliance, level 1—protocol state guarding, level 2—cryptographic and replay security checks, and level 3—functional flow execution) ensuring that higher-level logic is evaluated only after lower-level constraints are satisfied. The FSM is defined independently of any specific implementation, allowing the same formal model to support offline analysis and verification, as well as runtime inspection in monitoring and testing environments.

4.5. Formal Description of the Digital Twin Rule Engine

The formal rule system defined below serves two complementary purposes. First, it provides a precise, unambiguous specification of valid and invalid protocol behavior during the OTAA procedure. Second, it constitutes the executable logic of the Digital Twin, enabling runtime evaluation of incoming events against protocol semantics.

The rules are organized into hierarchical validation levels, reflecting the layered structure of the LoRaWAN protocol. Each level captures a distinct class of constraints, allowing violations to be localized to specific protocol layers and improving the interpretability of detected anomalies.

4.5.1. Notation and Semantics

In the following, we introduce the notation and formal semantics used to describe events, state transitions, and rule evaluation.

The Digital Twin is formalized as an event-driven FSM defined over event set $e \in \mathcal{E}$. According to the LoRaWAN specification [5], the event set is partitioned into message-related and timer-related events as

$$\mathcal{E} = \mathcal{E}_{\text{msg}} \cup \mathcal{E}_{\text{tmr}}. \tag{1}$$

Message event $e \in \mathcal{E}_{\text{msg}}$ is associated with an incoming frame m, which is characterized by attributes such as message (type), operating frequency (freq), data rate (dr), and cryptographic metadata. Timer events $e \in \mathcal{E}_{\text{tmr}}$ represent the expiration of previously scheduled protocol timers.

The system state is represented by the state vector

$$v \triangleq \langle state, history \rangle, \tag{2}$$

where *state* denotes the current protocol state and *history* captures the relevant communication context accumulated from prior events.

The communication context maintained by the Digital Twin captures all cryptographic and stateful parameters required by the LoRaWAN protocol. In particular, it includes a bounded history of DevNonce values generated by the end device and the AppNonce provided in the join-accept message, which jointly ensure freshness and replay protection of the join procedure. The context further stores the Application Key (AppKey), which acts as master cryptographic key in LoRaWAN and is used as input to the key derivation functions defined by the specification. From the AppKey, the Network Session Key (NwkSKey) and the Application Session Key (AppSKey) are derived upon successful completion of the join procedure and retained as part of the communication context for subsequent message integrity verification and payload encryption.

This communication context is actively maintained and updated by the Digital Twin and directly influences rule activation and admissible state transitions, rather than serving as a passive record of observed traffic.

State transitions are expressed as guarded rules of the form

$$(v, e) \xrightarrow{\text{rule}} v', \tag{3}$$

where e is the triggering event and v' denotes the updated state vector resulting from the application of the corresponding rule. A rule is enabled if its guard condition evaluates to true for the current state and event, in which case the corresponding state update is applied.

The rule engine processes incoming events using a hierarchical evaluation strategy. By structuring validation rules into successive levels, the Digital Twin mirrors the layered nature of the LoRaWAN protocol while enabling early detection of non-compliant traffic. Low-level rules capture fundamental radio and timing constraints, which are inexpensive to evaluate and allow malformed or physically implausible frames to be filtered before more complex analysis is performed. Higher-level rules operate on increasingly stateful and contextual information, incorporating security-oriented guardrails and protocol flow validation that require knowledge of prior events and system history. This design not only reduces unnecessary processing overhead but also improves the interpretability of detected anomalies by localizing violations to specific protocol layers. As a result, the proposed Digital Twin supports protocol-aware, explainable intrusion detection, while maintaining extensibility and alignment with LoRaWAN 1.0.3 specification.

In the following, we present the formal specification of the proposed rule system, structured according to its hierarchical evaluation levels.

4.5.2. States, Events, and Timers

In our Digital Twin state ranges over a finite set of protocol phases defined by the OTAA procedure. The explicitly modeled states and their definitions are listed in Table 1.

Table 1. FSM States of the LoRaWAN OTAA Digital Twin.

State	Description
NDEF	Idle or reset state, no active join procedure
JOINING_RX1DELAY	Waiting for the opening of the RX1 reception window
JOINING_RX1	RX1 reception window active
JOINING_RX2DELAY	Waiting for the opening of the RX2 reception window
JOINING_RX2	RX2 reception window active
JOINED	Device successfully activated and joined to the network

For compactness, we additionally employ state sets that represent semantically equivalent acceptance conditions listed in Table 2.

Table 2. Composite FSM states used for protocol guard conditions.

Composite State	Description
CAN_REQUEST_JOIN	Set of states in which a `Join Request` message is admissible
CAN_ACCEPT_JOIN	Set of states in which a `Join Accept` message may be processed

Message events correspond to the reception of LoRaWAN frames and carry a message type $m(e).\text{type} \in \{\text{JR} - \texttt{Join request}, \text{JA} - \texttt{Join Accept}, \text{DATA}\}$.

Timer events model protocol-defined timing boundaries and reception window timeouts. The OTAA model uses the timer identifiers as: TOUT_RX1START, TOUT_RX1END, TOUT_RX2START, TOUT_RX2END, which correspond to the opening and closing instants of the RX1 and RX2 receive windows. Timer expiration is denoted by $\text{Expired}(\cdot)$, while timer creation and removal are expressed via $\text{StartTimer}(\cdot)$ and $\text{DeleteTimers}(\cdot)$ in the transition rules.

4.5.3. Radio Layer Constraints (Level 0)

Level 0 rules validate radio-layer compliance independently of protocol history. These checks verify whether observed frames conform to region-specific frequency plans, data-rate mappings, duty-cycle limits, and reception-window parameters defined by the LoRaWAN Regional Parameters specification. In the present implementation, the EU868 profile is applied. Since radio parameters are inherently region-dependent, all Level 0 invariants are instantiated according to the selected regional configuration. The FSM-based Digital Twin itself remains region-agnostic and can be adapted to other LoRaWAN regions by substituting the corresponding parameter set without structural modification.

The first rule in Equation (4) illustrates well the general structure used throughout the specification. It should be read as follows: if an incoming event e corresponds to a `Join Request` message, and the device is in a state where a join request is admissible, and the message is received using a data rate different from the one the message is supposed to use according to the LoRaWAN specification, then the event is rejected and the system state remains unchanged. All subsequent rules follow the same guard action pattern, differing only in the specific conditions and actions applied.

$$\begin{aligned}
&(e \in \mathcal{E}_{\text{msg}} \wedge m(e).\text{type} = \text{JR} \wedge \mathit{state} \in \text{CAN_REQUEST_JOIN} \wedge m(e).\text{dr} \neq DR_{\text{JR}}) \\
&\quad \Rightarrow (v, e) \xrightarrow{\text{Reject_JR_FREQ}} v \\
&(e \in \mathcal{E}_{\text{msg}} \wedge m(e).\text{type} = \text{JR} \wedge \mathit{state} \in \text{CAN_REQUEST_JOIN} \wedge m(e).\text{freq} \neq F_{\text{JR}}) \\
&\quad \Rightarrow (v, e) \xrightarrow{\text{Reject_JR_FREQ}} v \\
&(e \in \mathcal{E}_{\text{msg}} \wedge m(e).\text{type} = \text{JA} \wedge \mathit{state} = \text{JOINING_RX1} \wedge m(e).\text{freq} \neq F_{\text{Up}}) \\
&\quad \Rightarrow (v, e) \xrightarrow{\text{Reject_JA_RX1_FREQ}} v \\
&(e \in \mathcal{E}_{\text{msg}} \wedge m(e).\text{type} = \text{JA} \wedge \mathit{state} = \text{JOINING_RX1} \wedge m(e).\text{dr} \neq DR_{\text{Up}}) \\
&\quad \Rightarrow (v, e) \xrightarrow{\text{Reject_JA_RX1_DR}} v \\
&(e \in \mathcal{E}_{\text{msg}} \wedge m(e).\text{type} = \text{JA} \wedge \mathit{state} = \text{JOINING_RX2} \wedge m(e).\text{freq} \neq F_{\text{JA_RX2}}) \\
&\quad \Rightarrow (v, e) \xrightarrow{\text{Reject_JA_RX2_FREQ}} v \\
&(e \in \mathcal{E}_{\text{msg}} \wedge m(e).\text{type} = \text{JA} \wedge \mathit{state} = \text{JOINING_RX2} \wedge m(e).\text{dr} \neq DR_{\text{JA_RX2}}) \\
&\quad \Rightarrow (v, e) \xrightarrow{\text{Reject_JA_RX2_DR}} v
\end{aligned} \tag{4}$$

Together, these rules ensure that only radio-compliant frames are forwarded to higher validation levels, preventing malformed or implausible traffic from influencing protocol state.

4.5.4. Flow Guard Constraints (Level 1)

Level 1 flow guard rules, presented in Equation (5) ensure that messages are processed only in semantically valid protocol states. These constraints prevent invalid message ordering, such as `Join Requests` or `Join Accept` messages appearing outside their designated protocol phases.

$$\begin{aligned}
&(e \in \mathcal{E}_{\text{msg}} \wedge m(e).\text{type} = \text{JR} \wedge \mathit{state} \notin \text{CAN_REQUEST_JOIN}) \\
&\quad \Rightarrow (v, e) \xrightarrow{\text{Reject_JR_STATE}} v \\
&(e \in \mathcal{E}_{\text{msg}} \wedge m(e).\text{type} = \text{JA} \wedge \mathit{state} \notin \text{CAN_ACCEPT_JOIN}) \\
&\quad \Rightarrow (v, e) \xrightarrow{\text{Reject_JA_STATE}} v
\end{aligned} \tag{5}$$

By enforcing state-dependent admissibility, these rules protect the Digital Twin from premature, delayed, or out-of-context protocol actions.

4.5.5. Security Constraints (Level 2)

Level 2 rules (see Equation (6) introduce security-oriented guardrails that leverage historical protocol context. These checks implement replay protection and cryptographic integrity validation by correlating incoming messages with previously observed events, such as DevNonce reuse or inconsistent gateway reception patterns.

$$\begin{aligned}
&(e \in \mathcal{E}_{\text{msg}} \land m(e).\text{type} = \text{JR} \land \text{DevNoncePresent}(m(e)) \land \text{ReplaySameGW}(m(e))) \\
&\quad \Rightarrow (v, e) \xrightarrow{\text{Reject_JR_REPLAY}} v \\
&(e \in \mathcal{E}_{\text{msg}} \land m(e).\text{type} = \text{JR} \land \text{DevNoncePresent}(m(e)) \land \text{SeenFromOtherGW}(m(e))) \\
&\quad \Rightarrow (v, e) \xrightarrow{\text{Info_JR_DEVNONCE}} v \\
&(e \in \mathcal{E}_{\text{msg}} \land m(e).\text{type} = \text{JR} \land \text{MICInvalid}(m(e))) \\
&\quad \Rightarrow (v, e) \xrightarrow{\text{Reject_JR_MIC}} v \\
&(e \in \mathcal{E}_{\text{msg}} \land m(e).\text{type} = \text{JA} \land \text{MICInvalid}(m(e))) \\
&\quad \Rightarrow (v, e) \xrightarrow{\text{Reject_JA_MIC}} v
\end{aligned} \tag{6}$$

Unlike purely statistical replay detection approaches, these rules explicitly encode protocol-defined freshness guarantees. This allows the model to distinguish between legitimate multi-gateway reception of the same frame and actual replay attempts or other activation-phase security violations.

4.5.6. State Transition Flow (Level 3)

Level 3 rules (Equations (7)–(13)) define the functional progression of the OTAA procedure itself. These rules are responsible for updating protocol state, managing timers associated with reception windows, and establishing session context upon successful activation.

$$\begin{aligned}
&(e \in \mathcal{E}_{\text{msg}} \land m(e).\text{type} = \text{JR} \land \mathit{state} \in \text{CAN_REQUEST_JOIN} \land \text{MICValid}(m(e))) \\
&\quad \Rightarrow (v, e) \xrightarrow{\text{JR_Allowed}} v' \\
&\quad \text{where } v' = \langle \text{JOINING_RX1DELAY}, \mathit{history}' \rangle \\
&\quad \land \mathit{history}' = \text{SaveDevNonce}(\mathit{history}, m(e)) \\
&\quad \land \text{StartTimer}(\text{JOINING}, \text{JA_TOUT_RX1START}) \\
&\quad \land \text{StartTimer}(\text{JOINING}, \text{JA_TOUT_RX1END}) \\
&\quad \land \text{StartTimer}(\text{JOINING}, \text{JA_TOUT_RX2START}) \\
&\quad \land \text{StartTimer}(\text{JOINING}, \text{JA_TOUT_RX2END})
\end{aligned} \tag{7}$$

$$\begin{aligned}
&(e \in \mathcal{E}_{\text{tmr}} \land \text{Expired}(\text{JA_TOUT_RX1START}) \land \mathit{state} = \text{JOINING_RX1DELAY}) \\
&\quad \Rightarrow (v, e) \xrightarrow{\text{RX1_START}} \langle \text{JOINING_RX1}, \mathit{history} \rangle
\end{aligned} \tag{8}$$

$$\begin{aligned}
&(e \in \mathcal{E}_{\text{tmr}} \land \text{Expired}(\text{JA_TOUT_RX1END}) \land \mathit{state} = \text{JOINING_RX1}) \\
&\quad \Rightarrow (v, e) \xrightarrow{\text{RX1_END}} \langle \text{JOINING_RX2DELAY}, \mathit{history} \rangle
\end{aligned} \tag{9}$$

$$\begin{aligned}
&(e \in \mathcal{E}_{\text{tmr}} \land \text{Expired}(\text{JA_TOUT_RX2START}) \land \mathit{state} = \text{JOINING_RX2DELAY}) \\
&\quad \Rightarrow (v, e) \xrightarrow{\text{RX2_START}} \langle \text{JOINING_RX2}, \mathit{history} \rangle
\end{aligned} \tag{10}$$

$$\begin{aligned}
&(e \in \mathcal{E}_{\text{tmr}} \land \text{Expired}(\text{JOINACCEPT_TOUT_RX2END}) \land \mathit{state} = \text{JOINING_RX2}) \\
&\quad \Rightarrow (v, e) \xrightarrow{\text{Reject_JA_TIMEOUT}} \langle \text{NDEF}, \mathit{history} \rangle
\end{aligned} \tag{11}$$

$$
\begin{aligned}
&(e \in \mathcal{E}_{\text{msg}} \land m(e).\text{type} = \text{JA} \land \mathit{state} \in \text{CAN\_ACCEPT\_JOIN} \land \text{MICValid}(m(e))) \\
&\quad \Rightarrow (v, e) \xrightarrow{\text{Accept\_JA}} v' \\
&\quad \text{where } v' = \langle \text{JOINED},\ \mathit{history}' \rangle \\
&\quad \land\ \mathit{history}' = \text{SaveJoinAccept}(\mathit{history}, m(e)) \\
&\quad \land\ \mathit{history}' = \text{DeriveSessionKeys}(\mathit{history}') \\
&\quad \land\ \text{DeleteTimers}(\text{JOINING})
\end{aligned} \tag{12}
$$

$$
\begin{aligned}
&(e \in \mathcal{E}_{\text{msg}} \land m(e).\text{type} = \text{DATA} \land \mathit{state} = \text{JOINED}) \\
&\quad \Rightarrow (v, e) \xrightarrow{\text{Valid\_DATA}} v
\end{aligned} \tag{13}
$$

This explicit modeling of timing windows and state transitions allows the Digital Twin to detect subtle violations of protocol behavior, such as delayed `Join Accept` messages or desynchronized reception attempts, which would remain invisible to packet-based detectors.

The generic fallback rule included in the implementation ensures total event handling by the rule engine but does not represent protocol behavior and is therefore excluded from the formal specification of the join procedure.

$$
\begin{aligned}
&(e \in \mathcal{E}) \\
&\quad \Rightarrow (v, e) \xrightarrow{\text{Reject\_UNHANDLED}} v
\end{aligned} \tag{14}
$$

For improved interpretability, a simplified set of states and transitions defined by the above rules is summarized in a graphical FSM representation in Figure 2. Blue transitions denote message-triggered events, while red transitions correspond to timer-triggered state changes. The diagram illustrates both nominal protocol progression and timing-driven deviations across RX1 and RX2 reception windows, including recovery paths and terminal failure states.

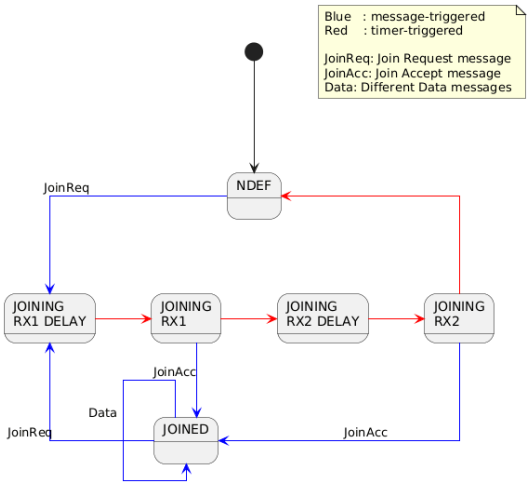

Figure 2. FSM representation of the LoRaWAN OTAA procedure as implemented by the Digital Twin. Blue transitions denote message-triggered events, while red transitions correspond to timer-triggered state changes associated with RX1 and RX2 reception windows.

5. Experimental Setup and System Implementation

This section describes the experimental environment and system implementation used to realize the Digital Twin-based protocol validation framework introduced in Section 4. The objective of the implementation is twofold: (i) to provide a realistic, reproducible LoRaWAN deployment that reflects operational constraints and (ii) to demonstrate how the formally specified FSM-based detection logic can be embedded into a practical, SOC-integrated monitoring pipeline.

5.1. LoRaWAN Testbed Environment

The experimental setup is based on a physical LoRaWAN network operated at Obuda University, complemented by cloud-hosted backend components. The testbed mirrors a typical real-world deployment in which resource-constrained end devices communicate with a centralized network server through multiple gateways, while security monitoring and analysis are performed at backend and SOC levels.

The LoRaWAN infrastructure consists of two RAK7246G WisGate Developer D0 gateways, each built on Raspberry Pi hardware and equipped with Semtech SX1308 concentrators. The gateways forward uplink traffic to an open-source ChirpStack LoRaWAN network server [28] deployed on virtual machines within the T-Systems Open Telekom Cloud (OTC). Backend services required by ChirpStack, including PostgreSQL, Redis, and an MQTT broker (Eclipse Mosquitto), are hosted within the same cloud environment.

End-device traffic is generated by commercial off-the-shelf LoRaWAN sensors (Seeed Studio SenseCAP S2101 and S2102), which periodically transmit environmental measurements such as temperature, humidity, light intensity, and battery level. In addition, programmable Wio-E5-based LoRaWAN clients are connected to Raspberry Pi boards via serial interfaces, allowing controlled experimentation with join procedures, key configurations, and device registration states.

The testbed operates exclusively on LoRaWAN version 1.0.3, reflecting its widespread adoption in production deployments and its default support in ChirpStack. This choice ensures both practical relevance and reproducibility.

5.2. Monitoring Points and Data Acquisition

To support protocol-aware intrusion detection, LoRaWAN communication is observed at multiple points along the communication pipeline as depicted in Figure 3. These monitoring points correspond to locations where protocol metadata, timing information, or session context can be accessed without interfering with normal operation.

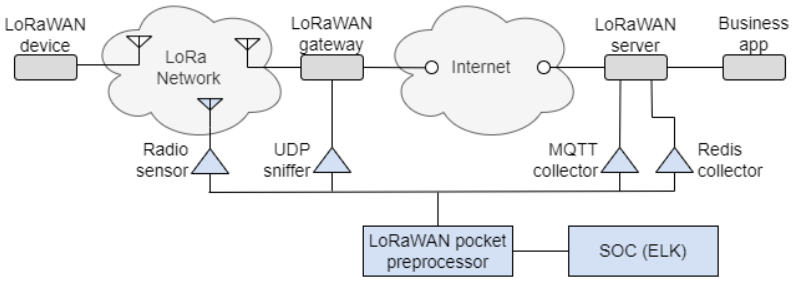

Figure 3. Main components of the LoRaWAN experimental test environment, illustrating end devices, gateways, backend services, and monitoring points (denoted with triangles) used for protocol validation. The setup reflects a realistic deployment in which protocol metadata is collected from multiple layers without interfering with normal network operation.

In the current implementation, two monitoring mechanisms are fully operational beyond the proof-of-concept level:

- MQTT-based monitoring: Decoded telemetry published by the gateway forwarders and the network server is captured by subscribing to relevant MQTT topics. Messages encoded in Protobuf format are decoded and transformed into structured JSON records.
- Redis-based frame log access: Internal frame logs maintained by the ChirpStack network server are retrieved from Redis streams using authenticated API access. These logs provide detailed, protocol-level context, including message types, device identifiers, and session-related metadata.

Additional monitoring approaches, such as radio capture and UDP interception between gateways and forwarders, are available in proof-of-concept form and are not required for the core operation of the Digital Twin. All collected telemetry is forwarded to the preprocessing layer for normalization and enrichment.

In a real-world deployment, the monitoring points can be positioned along the LoRaWAN communication path according to the architectural layer where observability is required. Radio-based monitoring can be deployed physically in proximity to gateways using LoRa Gateway hardwares with custom firmware [29] receivers, enabling passive bidirectional observation of uplink and downlink transmissions without interacting with the communication workflow.

UDP-level monitoring can be implemented on the link between gateways and the network server by means of traffic mirroring or port replication on the server-side interface. This placement provides visibility into forwarded frames while preserving the original packet forwarding path. As LoRaWAN deployments increasingly adopt LNS-based gateway protocols (e.g., LoRa Basics™ Station), monitoring strategies will correspondingly extend beyond the legacy Semtech UDP packet forwarder.

The remaining two monitoring points reside within the backend infrastructure, which is inherently part of LoRaWAN network operation. MQTT-based telemetry monitoring and Redis-based frame log access are integrated into the network server environment and rely on authenticated access to existing event streams and internal logs. These mechanisms are consistent with standard operational practices in managed LoRaWAN deployments.

From a functional perspective, a single monitoring point providing consistent protocol metadata is sufficient for the Digital Twin to perform deterministic state-transition validation. The FSM operates on observed message types, timing relationships, and protocol identifiers and therefore does not require simultaneous visibility across all layers to maintain state consistency. However, multi-layer monitoring increases observability completeness and reduces ambiguity in interpreting communication events. Correlating radio-layer attributes, forwarding metadata, and backend session context improves the robustness of anomaly interpretation and enables more precise localization of protocol inconsistencies across the communication pipeline.

All monitoring mechanisms operate on replicated telemetry data and do not perform inline packet interception or modification. Gateway scheduling, frame forwarding, and session establishment remain unaffected. In the implementation presented in this work, session context and key material are available within the operator-managed backend environment and are utilized to enable comprehensive protocol-conformance validation and integrity checks. This deployment model aligns with centralized LoRaWAN network management practices and preserves the integrity of production communication flows.

5.3. Preprocessing Pipeline and FSM Integration

All LoRaWAN telemetry passes through a dedicated preprocessing layer prior to ingestion into the SIEM. This layer is responsible for protocol normalization, metadata enrichment, and execution of the FSM-based Digital Twin described in Section 4.5.

The preprocessing pipeline is implemented as a modular, Python-based service that processes incoming events from heterogeneous sources (MQTT, Redis) in a unified manner. Events are transformed into a common internal representation that exposes protocol-relevant fields such as message type, timing metadata, radio parameters, and device identifiers, etc.

The FSM-based Digital Twin is embedded directly within this preprocessing layer and operates as an executable specification. Incoming events are evaluated against the hierarchical rule system (Levels 0–3), and protocol state is maintained on a per-device basis.

State transitions, timer scheduling, and security violations are handled locally within the preprocessor, ensuring that protocol semantics are validated before events reach the SIEM layer. The sequence diagram of the rule-based Digital Twin is depicted in Figure 4.

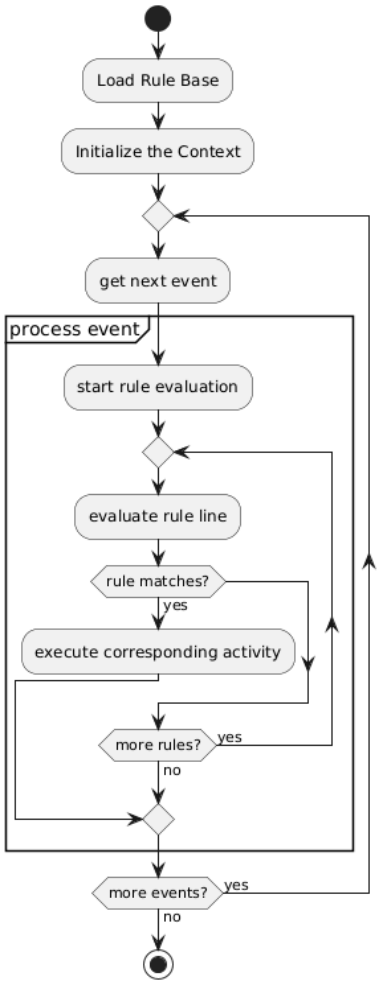

Figure 4. Sequence diagram illustrating the internal operation of the rule-based FSM engine embedded in the preprocessing layer. Incoming events are collected, evaluated against hierarchical protocol rules, used to update the communication context, and emitted as structured detection records toward the SIEM layer.

Our implementation allows the preprocessing layer to operate in the operator-managed backend infrastructure mode, where access to AppKeys enables session key derivation, MIC validation, and payload decryption, allowing full protocol field analysis.

Figure 5 presents the end-to-end operation of the preprocessing pipeline, highlighting how protocol-aware validation is integrated before SIEM ingestion. Telemetry streams collected from gateways and backend components are ingested by the preprocessing service, where events are decoded, enriched, and mapped into a unified internal representation. The embedded FSM-based Digital Twin evaluates each event in sequence, maintaining per-device protocol state and applying the hierarchical rule set. Detection outcomes, including state transitions, rule violations, and timing-related events, are emitted as structured records and forwarded to the SIEM layer. By performing protocol validation prior to SIEM ingestion, the pipeline ensures that downstream security analytics operate on semantically enriched, state-aware events rather than raw or decontextualized logs. This design preserves protocol semantics while remaining compatible with standard SOC workflows.

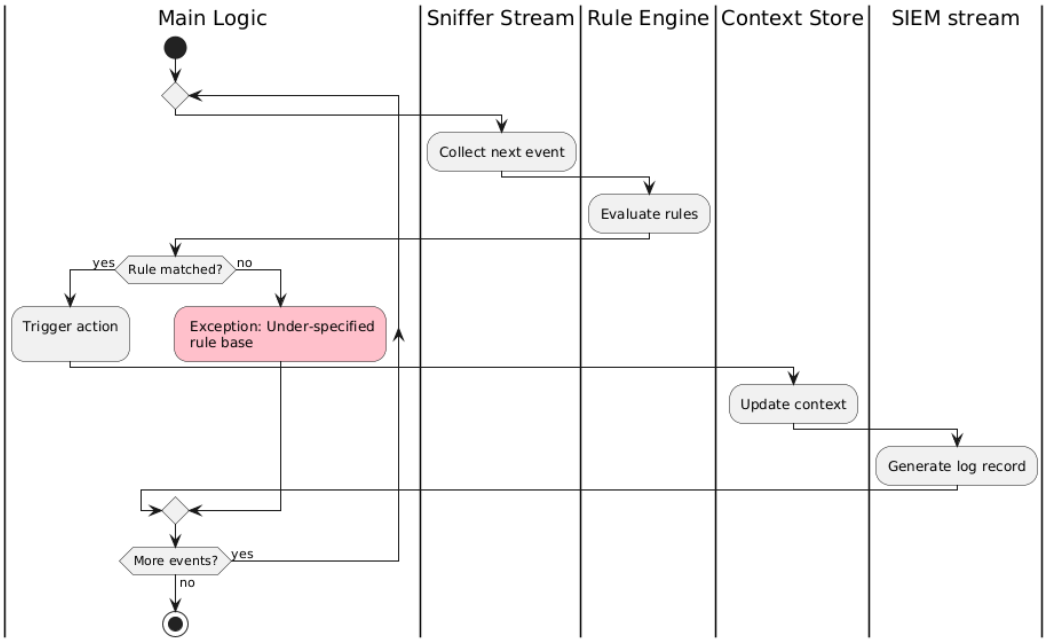

Figure 5. End-to-end preprocessing pipeline integrating Digital Twin-based protocol validation into SOC workflows. Telemetry streams from gateways and backend components are decoded, normalized, and enriched before being evaluated by the FSM. Detection outcomes, including state transitions and rule violations, are forwarded to the SIEM as semantically enriched events.

5.4. SIEM Integration and Visualization

Processed events and detection outcomes generated by the preprocessing layer are forwarded to an ELK-based [30] SIEM infrastructure. Elasticsearch is used for storage and indexing, while Kibana provides dashboards and alerting mechanisms for SOC analysts.

FSM state transitions, rule violations, and contextual metadata are mapped to structured fields, enabling correlation across devices, gateways, and time windows. This integration allows protocol-level anomalies, such as invalid join sequences, nonce reuse, timing violations, or inconsistent gateway metadata, to be visualized and acted upon within standard SOC workflows.

Importantly, the SIEM layer remains decoupled from protocol logic. All protocol-aware reasoning is confined to the preprocessing layer, while the SIEM focuses on aggregation, visualization, and alert management. This separation of concerns improves maintainability and allows the Digital Twin logic to evolve independently of the SOC tooling.

6. Results

To demonstrate the operational behavior of the proposed Digital Twin-based protocol validator and its SIEM-facing outputs, we executed a set of controlled OTAA interaction scenarios in our LoRaWAN test environment. The scenarios, summarized in Table 3, cover both nominal protocol execution (S1) and representative deviations involving nonce reuse (S2, S8), timing (S5, S6, S7), radio parameter violations (S4), and cryptographic integrity failures (S3, S9). For each scenario, the Digital Twin-based protocol validator produces a deterministic FSM outcome and a corresponding SIEM-level alert classification. Each experiment generates a stream of normalized events that is processed by the preprocessing layer where the FSM-based Digital Twin maintains per-device protocol state and emits structured detection outcomes before SIEM ingestion.

Table 3. OTAA protocol compliance and anomaly detection scenarios evaluated using the Digital Twin-based protocol validator.

ID	Scenario Description	Deviation Type	FSM Final State	Detection Level	SIEM Outcome
S1	Join-Request → Join-Accept → Data	None (baseline)	JOINED	–	No alert
S2	Join-Request with reused DevNonce observed from another gateway, followed by Join-Accept and Data	DevNonce reuse (multi-GW)	JOINED	L2 (Sec)	Notice
S3	Join-Request with invalid MIC	MIC integrity violation	NDEF	L2 (Sec)	Reject
S4	Join-Request transmitted on invalid frequency	Radio parameter violation	NDEF	L0 (Radio)	Reject
S5	Join-Request → timeout → new Join-Request (new DevNonce) → Join-Accept → Data	Timing deviation with recovery	JOINED	L3 (Flow)	Notice
S6	Join-Request → Join-Accept received only in RX2 window	RX1 window miss	JOINED	L3 (Timing)	Notice
S7	Join-Request → Join-Accept received after RX2 window	RX window violation	NDEF	L3 (Timing)	Reject
S8	Join-Request → timeout → repeated Join-Request with identical DevNonce	Replay after timeout	NDEF	L2 (Sec)	Reject
S9	Join-Request → Join-Accept with invalid MIC	MIC integrity violation	NDEF	L2 (Sec)	Reject
S10	Join-Request → Join-Accept transmitted on invalid frequency	Radio parameter violation	JOINING	L0 (Radio)	Reject
S11	Join-Accept transmitted in non-admissible state	State violation	unchanged	L1 (Flow Guard)	Reject

6.1. FSM State Evolution and Detection Outcomes

Figure 6 illustrates a representative excerpt from the SIEM event stream preprocessed by the FSM-based protocol validator during OTAA execution. Each entry corresponds to a protocol-relevant event processed by the Digital Twin.

The fields prevState and newState capture the Digital Twin's current protocol context and its evolution, while MSG.Type indicates whether the triggering event is a protocol message (e.g., JOIN_REQUEST, JOIN_ACCEPT, UNCONFIRMED_DATA_UP) or an internal timer event (shown as null in the message type field). The action field provides a human-readable explanation of the decision (e.g., Valid `JoinRequest`, `Join Accept` RX1 timeout, Duplicate DevNonce from other GW, MIC invalid), and the icons in the 3rd column encodes the alert severity (e.g., informational/notice vs. reject). This visualization demonstrates a key operational property: the protocol validator is not limited to "packet verdicts" but produces explainable, stateful protocol reasoning that is directly consumable by SOC workflows.

The first sequence (lines 1–5) in the figure demonstrates a failed join attempt caused by a `Join Accept` timeout. Following a valid `Join Request`, the FSM transitions through the RX1 and RX2 reception windows. As no valid `Join Accept` is observed within the protocol-defined timing constraints, the Digital Twin triggers a timeout rule at the end of the RX2 window, transitioning the FSM into the terminal `NDEF` state. This behavior corresponds to scenario S7 in Table 3 and illustrates how timing violations are deterministically enforced by the protocol model.

Table 4 links the structured test scenarios of Table 3 to the concrete execution trace shown in Figure 6. By explicitly referencing log row numbers, the incremental semantic contribution of each validation level becomes empirically observable. Row 22 (S10) demonstrates that purely regional frequency violations are isolated at Level 0 without protocol-state knowledge. S11 shows that out-of-context message ordering requires OTAA state-awareness (Level 1). Row 12 (S4) and Row 7 (S6) illustrate that integrity and historical

correlation checks emerge only at Level 2. Finally, Row 23 (S7) confirm that deterministic RX-window timing and data messages in a not-valid state require the explicit temporal modeling of Level 3. This trace-based mapping demonstrates that higher semantic layers provide orthogonal detection capability rather than redundant checks.

#	timestamp	...	action	event	MSG type	prevState	newState
1	16:25:41.482	✓	Valid JoinRequest	Message	JOIN_REQUEST	NDEF	JOINING_RX1DELAY
2	16:25:46.442	✓	Starting RX1 window	Timer	(null)	JOINING_RX1DELAY	JOINING_RX1
3	16:25:47.432	i	Join Accept RX1 timeout	Timer	(null)	JOINING_RX1	JOINING_RX2DELAY
4	16:25:47.442	✓	Starting RX2 window	Timer	(null)	JOINING_RX2DELAY	JOINING_RX2
5	16:25:48.432	✗	Join Accept timeout	Timer	(null)	JOINING_RX2	NDEF
6	16:25:50.482	✓	Valid JoinRequest	Message	JOIN_REQUEST	NDEF	JOINING_RX1DELAY
7	16:25:50.682	i	Duplicate DevNonce from other GW	Message	JOIN_REQUEST	JOINING_RX1DELAY	JOINING_RX1DELAY
8	16:25:55.442	✓	Starting RX1 window	Timer	(null)	JOINING_RX1DELAY	JOINING_RX1
9	16:25:55.482	✗	Data Message invalid state	Message	UNCONFIRMED_D...	JOINING_RX1	JOINING_RX1
10	16:25:55.782	✓	Valid JoinAccept in First Rec Window	Message	JOIN_ACCEPT	JOINING_RX1	JOINED_GRACE_P
11	16:25:56.782	✓	Second Join Accept during the Grace P...	Message	JOIN_ACCEPT	JOINED_GRACE_P	JOINED
12	16:25:58.482	✗	MIC invalid	Message	JOIN_REQUEST	JOINED	JOINED
13	16:26:00.482	✓	Valid JoinRequest	Message	JOIN_REQUEST	JOINED	JOINING_RX1DELAY
14	16:26:05.442	✓	Starting RX1 window	Timer	(null)	JOINING_RX1DELAY	JOINING_RX1
15	16:26:06.432	i	Join Accept RX1 timeout	Timer	(null)	JOINING_RX1	JOINING_RX2DELAY
16	16:26:06.442	✓	Starting RX2 window	Timer	(null)	JOINING_RX2DELAY	JOINING_RX2
17	16:26:06.782	✓	Valid JoinAccept in Second Rec Window	Message	JOIN_ACCEPT	JOINING_RX2	JOINED
18	16:26:08.482	✓	Valid JoinRequest	Message	JOIN_REQUEST	JOINED	JOINING_RX1DELAY
19	16:26:13.442	✓	Starting RX1 window	Timer	(null)	JOINING_RX1DELAY	JOINING_RX1
20	16:26:14.432	i	Join Accept RX1 timeout	Timer	(null)	JOINING_RX1	JOINING_RX2DELAY
21	16:26:14.442	✓	Starting RX2 window	Timer	(null)	JOINING_RX2DELAY	JOINING_RX2
22	16:26:14.782	✗	JoinAccept Frequency not allowed	Message	JOIN_ACCEPT	JOINING_RX2	JOINING_RX2
23	16:26:15.432	✗	Join Accept timeout	Timer	(null)	JOINING_RX2	NDEF

Figure 6. Kibana-based SIEM visualization of FSM evaluation results during OTAA execution. Each log entry corresponds to a protocol-relevant event processed by the Digital Twin, showing previous and new FSM states and triggering message or timer events and rule-level decisions. The visualization demonstrates how protocol semantics are exposed directly to SOC analysts. A green ✓ indicates a protocol transition consistent with the LoRaWAN specification, a blue "i" symbol denotes an informational event that may be acceptable depending on context, while a red cross marks a deviation that violates the protocol specification.

Table 4. Incremental detection capabilities of validation Levels 0–3, empirically demonstrated through the executed scenarios listed in Table 3. Deviations are linked to observed SIEM traces (Figure 6 row numbers) where applicable. A checkmark (✓) indicates that the deviation can be detected at the corresponding validation level, whereas "–" denotes that the given level does not provide the respective detection capability.

Table 3	Figure 6 Row	Scenario Description	L0	L1	L2	L3	Incremental Semantic Contribution
S10	22	Join-Request → Join-Accept transmitted on invalid frequency	✓	–	–	–	Pure regional constraint violation (radio-level).
S11	–	Join-Accept transmitted in non-admissible state	–	✓	–	–	Flow-guard admissibility: rejects out-of-context JoinAccept even if radio-level compliant.
S9	12	Join-Request → Join-Accept with invalid MIC	–	–	✓	–	Cryptographic integrity failure; syntactically valid at lower layers.
S2	7	Join-Request with reused DevNonce observed from another gateway (multi-GW)	–	–	✓	–	Requires historical and cross-gateway correlation.
S7	23	Join-Request → Join-Accept received after RX2 windows	–	–	–	✓	Timer-driven RX window semantics; requires explicit timing model.

6.2. Handling of Benign Deviations and Recovery

The subsequent sequence (lines 6–11) demonstrates a recovery scenario in which a new `Join Request` is issued after a failed activation attempt. When a valid `Join Request` with a fresh DevNonce is received, the FSM correctly re-enters the join procedure and transitions through the RX1 and RX2 windows. Upon successful reception of a valid `Join Accept` message, the FSM reaches the `JOINED` state, after which normal data traffic is accepted.

6.3. Detection of Suspicious but Tolerated Behavior

Figure 6 also highlights a scenario in which a duplicate DevNonce is observed from a different gateway during the join procedure (line 7). In this case, the Digital Twin raises an informational notice while maintaining the current FSM state, allowing the join procedure to continue. This behavior corresponds to scenario S2 in Table 3 and reflects a deliberate design choice: the validator distinguishes between protocol violations that require immediate rejection and suspicious conditions that warrant monitoring without disrupting legitimate device activation.

6.4. Cryptographic and Protocol-Level Enforcement

Cryptographic integrity violations are consistently identified and annotated at the security validation level. As shown in the log excerpt, `Join Request` messages with invalid MIC values (line 12) are rejected without triggering state transitions, ensuring that malformed or forged messages cannot influence protocol state. Similar action applies to `Join Accept` messages failing integrity verification, as reflected by transitions to the `NDEF` state in scenarios S3 and S9.

6.5. SIEM-Level Interpretability

Importantly, all detection outcomes generated by the Digital Twin are propagated to the SIEM layer as structured, interpretable events. Each alert includes the FSM state, the triggered rule level, and the specific rule identifier responsible for the decision. This enables SOC analysts to trace alerts back to protocol semantics rather than opaque anomaly scores, improving explainability and operational trust.

Beyond per-event explanations, the structured fields enable aggregation and dashboarding: analysts can query, for example, all Level 2 integrity failures, or all RX-window-related notices, and correlate them with gateway context and radio metadata. This makes protocol violations measurable at the SOC scale and supports both real-time alerting and retrospective incident analysis.

6.6. FSM-Based End-Device State Evolution

Figure 7 provides a complementary, device-centric view of the Digital Twin-based intrusion detection process by visualizing the state evolution of an end device throughout successive OTAA interactions. While the SIEM excerpts in Figure 6 focus on backend-level event streams and rule evaluations, this figure reconstructs the logical protocol trajectory as maintained by the FSM for a single device instance.

The visualization highlights how the Digital Twin transitions deterministically between protocol states (e.g., NDEF, JOINING_RX1, JOINING_RX2, JOINED) in response to both message arrivals and timer expirations. Failed activation attempts, recovery through fresh join requests, and successful session establishment are represented as explicit state transitions rather than implicit side effects of packet handling.

This representation makes protocol-level deviations, such as prolonged residence in joining states, repeated activation attempts, or timing-driven fallbacks directly observable. By exposing the end-device lifecycle as a sequence of well-defined states, the figure illus-

trates a key advantage of the proposed approach: protocol misuse and anomalous behavior are not inferred indirectly from traffic statistics but are identified through violations of expected state progression. This device-centric perspective reinforces the interpretability of the Digital Twin model and complements the SIEM-oriented results by linking backend detection outcomes to concrete protocol behavior.

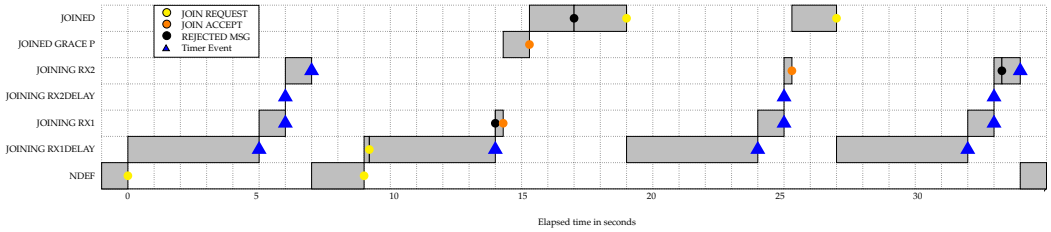

Figure 7. Temporal evolution of an end device's protocol state as maintained by the FSM-based Digital Twin during successive OTAA interactions. State transitions are driven by both message arrivals and timer expirations, illustrating failed join attempts, recovery through fresh Join Requests, and successful activation.

7. Discussion

This work demonstrates that protocol-aware validation layer for LoRaWAN can be realized as an operationally deployable Digital Twin that is both state-synchronized and SIEM-native. In contrast to detectors that operate on isolated packets or coarse correlation rules, the proposed approach continuously maintains the OTAA execution context and evaluates events against an executable specification, yielding decisions that are inherently explainable through explicit state transitions and rule-level attribution.

7.1. What the Approach Adds Beyond Conventional LoRaWAN Monitoring—Comparative Positioning

A recurring limitation in LoRaWAN SOC practice is that many security-relevant conditions are only meaningful when interpreted in the correct protocol phase. The presented results illustrate that anomalies such as RX-window violations, repeated join attempts, or suspicious DevNonce reuse cannot be reliably interpreted without a stateful model of expected progression. By embedding the FSM-based Digital Twin directly into the preprocessing pipeline (prior to SIEM ingestion), the system can attach protocol semantics to telemetry early, ensuring that downstream analytics do not operate on ambiguous, decontextualized events.

A second contribution is the layered decision structure (Levels 0–3), which enables operationally useful differentiation between (i) radio/configuration issues, (ii) protocol sequencing violations, (iii) security-integrity and replay signals, and (iv) timing-driven flow outcomes. This separation supports SOC triage and alert tuning: e.g., noisy radio-level misconfigurations can be handled differently from cryptographic integrity failures.

Table 5 summarizes the key differences. In contrast to traffic-based or statistics-based approaches, the proposed FSM-based Digital Twin provides explicit state modeling and protocol-phase-aware validation, resulting in structured and interpretable SOC-level outputs.

Table 5. Comparative positioning of LoRaWAN protocol-aware validation layer.

Feature	Traffic-Based IDS	Statistical IDS	Proposed FSM-Based Digital Twin
Deployment assumptions	Traffic visibility only	Aggregated statistics	Multi-layer telemetry; optional session context
Protocol awareness	No	Partial	Specification-level
State modeling	None	Implicit/statistical	Explicit finite state machine
Timing reasoning (RX1/RX2)	No	Threshold-based	Rule validated timing semantics
Replay handling	Signature/volumetric	Statistical DevNonce anomaly	Context-aware nonce history
Cryptographic validation	Not available	Not available	Integrated (when session context is available)
Explainability	Low	Medium (feature-based reasoning)	High (state-transition traceability)

7.2. Scalability, Runtime Characteristics, and Operational Considerations

The proposed Digital Twin maintains an independent finite-state context per monitored device. Memory complexity therefore scales linearly with the number of active devices (O(N)), where each device state consists of a bounded set of protocol attributes (current FSM state, nonce history, timing references, and minimal session context). No global state synchronization across devices is required.

Processing complexity is event-driven and constant per message. Each observed event triggers a deterministic state-transition evaluation with fixed guard-condition checks derived from the protocol specification. Consequently, computational overhead scales linearly with the message rate rather than quadratically with device count.

The FSM does not perform continuous polling or cross-device correlation, further bounding runtime overhead.

To complement the theoretical complexity analysis, we performed controlled runtime measurements of the FSM-based preprocessing layer under repeated OTAA-related event processing. The experiments were conducted on a virtual machine deployed in the Open Telekom Cloud (OTC) EU-DE region. The instance used a general-purpose configuration (s2.medium.4) with 1 vCPU and 4 GiB of RAM and was based on the public image Standard_Ubuntu_24.04_amd64_uefi_latest. In total, 10,750 protocol-relevant events were evaluated, including 3500 Join Requests, 2250 Join Accept messages, and 5000 timer-triggered transitions.

The measured average processing latency per event remained in the sub-millisecond range across all event types. Join Request evaluation required on average 0.270 ms, Join Accept processing required 0.313 ms, while timer-driven state transitions required 0.052 ms. The overall mean processing time across all event categories was 0.178 ms per event. The empirical latency distribution illustrated in Figure 8 confirms that the majority of events were processed within $\leq$0.3 ms. The 95th percentile remained below 0.4 ms with no evidence of systematic latency amplification under repeated state transitions.

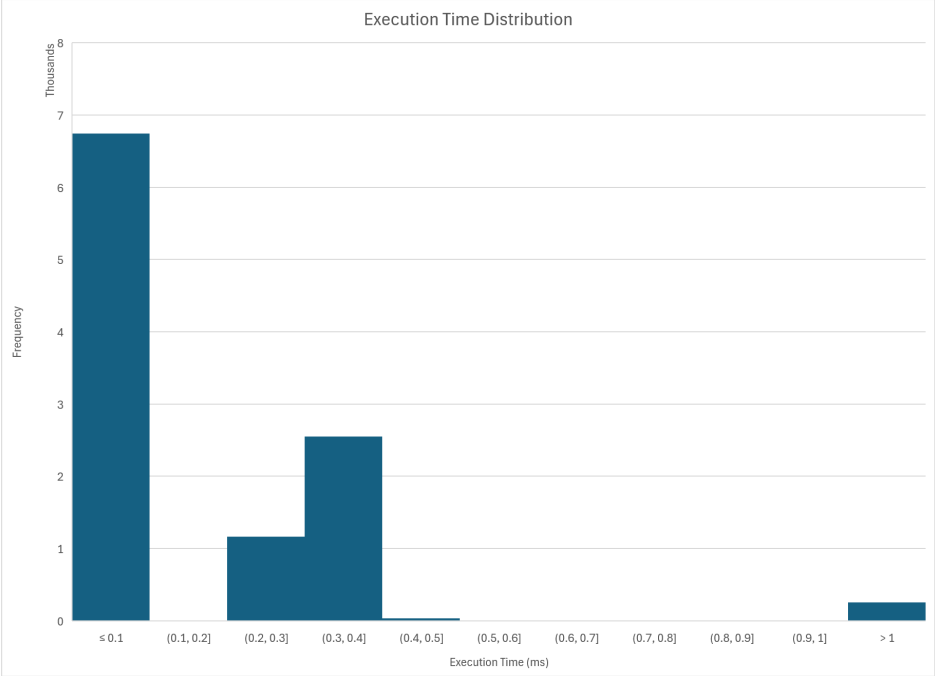

Figure 8. Distribution of per-event processing latency measured during evaluation of 10,750 OTAA-related events. The histogram illustrates that the majority of FSM evaluations completed within sub-millisecond time, confirming bounded and stable per-event runtime behavior.

Memory consumption per tracked device remained stable during evaluation. The average memory allocation associated with FSM processing was approximately 30 kB per device context (29,055 bytes for Join Accept, 30,497 bytes for Join Request handling, and 30,194 bytes for timer events), with negligible variance between event types. Since each device maintains an independent and bounded state vector (current state, nonce history, timing references, minimal session context), memory scales linearly with the number of concurrently tracked devices.

These empirical results substantiate the earlier analytical claims: (i) per-event processing cost remains constant and bounded, (ii) runtime scales linearly with message rate, and (iii) per-device memory footprint remains modest and predictable, supporting deployment in resource-constrained backend environments.

From an operational perspective, the Digital Twin functions as a preprocessing layer that augments telemetry with state-aware protocol context before ingestion into the SIEM environment. Instead of requiring downstream components to reconstruct protocol semantics from raw events, the SIEM receives structured validation outcomes enriched with state progression information and explanatory context. This reduces analytical ambiguity and supports faster decision-making and more precise filtering of security-relevant events.

While this preprocessing does not eliminate the general scalability challenges inherent to SIEM-based monitoring infrastructures, it shifts part of the semantic reconstruction burden to a deterministic, protocol-aware layer. As a result, higher-level correlation, statistical analysis, and machine-learning based detection can operate on semantically enriched inputs. A comprehensive quantitative assessment of large-scale SOC performance impact remains part of future work.

7.3. Impact of Non-Ideal Network Conditions

LoRaWAN operation over a shared wireless medium inherently involves packet loss and variable reception conditions. The FSM-based validation layer is therefore designed to distinguish between specification-compliant retry behavior and true protocol violations.

In the OTAA procedure, missing Join Accept messages result in a timeout-driven transition back to the idle (NDEF) state. A subsequent Join Request with a fresh DevNonce is processed as a new, valid activation attempt and does not trigger a rejection. This behavior was observed in scenario S5 (Table 3), where timeout-induced recovery leads to successful activation without false-positive security events.

Repeated Join Requests using the same DevNonce after a timeout are rejected only when violating replay-protection semantics (scenario S8). In contrast, multi-gateway reception of identical frames does not trigger rejection but is annotated at Level 2 as contextual information (scenario S2). These results indicate that retry patterns consistent with LoRaWAN v1.0.3 do not cause false rejects. Rejection decisions are tied to explicit violations of state progression, nonce freshness, or cryptographic integrity rather than to isolated packet loss.

A comprehensive stress evaluation under high-loss or large-scale concurrency conditions remains for future work.

7.4. Scope and Evaluation Limitations

The current implementation and evaluation focuses on the OTAA procedure and a representative set of deviations around join integrity, replay signals, and RX-window timing. While these cover a security-critical phase of LoRaWAN, the full attack surface also includes post-join communication behaviors such as FCnt anomalies, ADR misuse, MAC command abuse, downlink scheduling patterns, and device-class-specific timing semantics.

7.5. Future Directions

A natural extension of the proposed approach is to expand the FSM state space beyond the OTAA procedure to cover the complete LoRaWAN device lifecycle, including uplink and downlink exchanges, frame counter validation strategies, MAC command sequences, and class-dependent receive behavior.

The methodology itself is not tightly coupled to a specific LoRaWAN specification version. While the current implementation targets LoRaWAN 1.0.3, adaptation to LoRaWAN 1.1 would primarily require incorporation of rejoin procedures, Join Server interactions, and updated key-derivation semantics.

In mixed deployments, this extension can be realized in a structurally transparent manner. The network server (e.g., ChirpStack) maintains a device profile for each end-device, including the supported LoRaWAN version. The preprocessor layer can retrieve (or cache) this version information and deterministically route incoming messages to a version-specific FSM instance. Accordingly, parallel FSM models may operate for LoRaWAN 1.0.x and LoRaWAN 1.1 devices, with routing decisions remaining static per device. While the 1.1 specification introduces additional state transitions (e.g., RejoinReq types and Join Server-mediated key updates), these modify the internal state space but do not alter the layered validation concept. Thus, heterogeneous 1.0.x/1.1 environments can be supported under a unified, version-aware FSM-based validation framework.

Beyond LoRaWAN, the Digital Twin abstraction can serve as a blueprint for protocol-aware validation in other LPWAN and IoT communication systems with explicit stateful operation and timing constraints.

Finally, the framework lends itself to hybrid designs combining formal state models with statistical or learning-based methods. Structured outputs such as state trajectories, rule activations, and timing residuals may serve as interpretable features for anomaly-scoring mechanisms in extended SOC architectures.

In addition, the current evaluation is scenario-based and conducted under controlled experimental conditions. Large-scale deployment aspects—including multi-device concurrency, sustained benign churn (e.g., repeated joins due to poor coverage), and long-term alert-rate characterization have not yet been empirically quantified. Operational parameters such as configuration effort, alert-volume management, and sustained SOC maintenance therefore require extended field validation beyond the scope of this study.

8. Conclusions

This paper introduced a protocol-aware validation methodology for LoRaWAN networks based on a finite-state Digital Twin formalizing the OTAA procedure. By modeling expected activation message sequences, timing constraints, and state transitions, the proposed approach enables continuous validation of observed communication against protocol semantics rather than relying solely on statistical indicators or signature-based mechanisms.

Through controlled OTAA scenarios executed in a realistic LoRaWAN testbed, we demonstrated that the FSM-based Digital Twin produces deterministic and interpretable state-conformance outcomes that can be directly integrated into SOC-oriented telemetry pipelines. Security-relevant deviations—such as replay behavior, timing inconsistencies, and integrity anomalies—manifest as explicit violations of protocol state progression when protocol context and, where available, cryptographic session information are incorporated.

Importantly, the presented framework does not constitute a complete IDS in isolation. Instead, it establishes a state-synchronized protocol-validation layer that can serve as a foundational component of a protocol-aware IDS architecture. By enriching telemetry with structured, semantically grounded context prior to SIEM ingestion, the approach

enables downstream statistical, correlation-based, or machine learning-driven detection mechanisms to operate on protocol-consistent, explainable inputs.

While the current implementation focuses on the OTAA phase, the methodology is extensible to post-join communication behavior and broader lifecycle monitoring. In this sense, the work represents a concrete step toward protocol-aware intrusion detection for LoRaWAN networks, demonstrating how formal state modeling can systematically bridge protocol specification and operational SOC analytics in constrained IoT environments.

Author Contributions: Conceptualization, Z.B., R.F. and E.K.; methodology, Z.B., R.F. and E.K.; formal modeling, Z.B., R.F. and E.K.; software, Z.B.; investigation, Z.B., R.F. and E.K.; writing, Z.B., R.F. and E.K.; visualization, Z.B. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The reference implementation of the FSM-based Digital Twin and the associated preprocessing components is available as open-source software at https://github.com/zsoltbringye/LoRa_FSM, (accessed on 10 January 2026) supporting reproducibility.

Acknowledgments: This research was supported by access to the OTC cloud service provided by Deutsche Telekom TSI Hungary Ltd. We gratefully acknowledge the infrastructure support, which contributed to the results presented in this publication and supports the promotion of TSI OTC both nationally and internationally. During the preparation of this manuscript, the authors used ChatGPT (version 5.2) to support language refinement and improve textual clarity. The authors have reviewed and edited the output and take full responsibility for the content of this publication.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Tightiz, L.; Dang, L.M.; Padmanaban, S.; Hur, K. Metaverse-driven smart grid architecture. *Energy Rep.* **2024**, *12*, 2014–2025. [CrossRef]
2. Kenyeres, M.; Kenyeres, J.; Hassankhani Dolatabadi, S. Distributed consensus gossip-based data fusion for suppressing incorrect sensor readings in wireless sensor networks. *J. Low Power Electron. Appl.* **2025**, *15*, 6. [CrossRef]
3. Tolvaly-Roşca, F.; Fekete, G.; Bíró, I.; Fekete, A.Z.; Forgó, Z. Real-Time IoT Solution to Monitor and Control dMVHR Units in Real-Life Environment. *Acta Polytech. Hung.* **2024**, *21*, 303–322. [CrossRef]
4. Pekarčík, P.; Chovancová, E.; Chovanec, M.; Štancel, M. A Centralized Approach to Intrusion Detection System Management: Design, Implementation and Evaluation. *Acta Polytech. Hung.* **2025**, *22*, 7–26. [CrossRef]
5. LoRa Alliance. LoRaWAN Specification v1.0.3. 2018. Available online: https://lora-alliance.org/resource_hub/lorawan-specification-v1-0-3/ (accessed on 10 January 2026).
6. Semtech Corporation. LoRa Technology Overview. n.d. Available online: https://www.semtech.com/lora (accessed on 10 January 2026).
7. LoRa Alliance. LoRaWAN Specification v1.1. 2017. Available online: https://lora-alliance.org/resource_hub/lorawan-specification-v1-1/ (accessed on 10 January 2026).
8. Bringye, Z.; Fleiner, R.; Umhauser, B.; Aradi, M.; Kail, E. Extending SOC Capabilities to LoRaWAN: A Cloud-Integrated Intrusion Detection Framework for IoT Networks. *Acta Polytech. Hung.* **2026**, *23*, 65–84. [CrossRef]
9. Butun, I.; Pereira, N.; Gidlund, M. Security Risk Analysis of LoRaWAN and Future Directions. *Future Internet* **2018**, *11*, 3. [CrossRef]
10. Eldefrawy, M.; Butun, I.; Pereira, N.; Gidlund, M. Formal security analysis of LoRaWAN. *Comput. Netw.* **2019**, *148*, 328–339. [CrossRef]
11. Hessel, F.; Almon, L.; Hollick, M. LoRaWAN Security: An Evolvable Survey on Vulnerabilities, Attacks and their Systematic Mitigation. *ACM Trans. Sens. Netw.* **2023**, *18*, 70. [CrossRef]
12. Loukil, S.; Fourati, L.; Nayyar, A.; Chee, K.W.A. Analysis of LoRaWAN 1.0 and 1.1 Protocols Security Mechanisms. *Sensors* **2022**, *22*, 3717. [CrossRef] [PubMed]
13. Tomasin, S.; Zulian, S.; Vangelista, L. Security analysis of lorawan join procedure for internet of things networks. In *2017 IEEE Wireless Communications and Networking Conference Workshops (WCNCW)*; IEEE: Piscataway, NJ, USA, 2017; pp. 1–6.

14. Spadaccino, P.; Cuomo, F. Intrusion Detection Systems for IoT: Opportunities and challenges offered by Edge Computing and Machine Learning. *arXiv* **2022**, arXiv:2012.01174. [CrossRef]
15. Verzegnassi, E.G.M.; Tountas, K.; Pados, D.A.; Cuomo, F. Data conformity evaluation: A novel approach for IoT security. In *2019 IEEE 5th World Forum on Internet of Things (WF-IoT)*; IEEE: Piscataway, NJ, USA, 2019; pp. 842–846.
16. Liao, H.J.; Lin, C.H.R.; Lin, Y.C.; Tung, K.Y. Intrusion detection system: A comprehensive review. *J. Netw. Comput. Appl.* **2013**, *36*, 16–24. [CrossRef]
17. Cantelli-Forti, A.; Colajanni, M.; Russo, S. Penetrating the Silence: Data Exfiltration in Maritime and Underwater Scenarios. In *IEEE 48th Conference on Local Computer Networks (LCN)*; IEEE: Piscataway, NJ, USA, 2023; pp. 1–6. [CrossRef]
18. Danish, S.M.; Nasir, A.; Qureshi, H.K.; Ashfaq, A.B.; Mumtaz, S.; Rodriguez, J. Network Intrusion Detection System for Jamming Attack in LoRaWAN Join Procedure. In *2018 IEEE International Conference on Communications (ICC)*; IEEE: Piscataway, NJ, USA , 2018; pp. 1–6. [CrossRef]
19. Horák, T.; Střelec, P.; Kováč, S.; Tanuška, P.; Nemlaha, E. Detection of IoT communication attacks on LoRaWAN gateway and server. In *Computer Science On-Line Conference*; Springer: Berlin/Heidelberg, Germany, 2023; pp. 489–497.
20. Fu, Y.; Yan, Z.; Cao, J.; Koné, O.; Cao, X. An Automata Based Intrusion Detection Method for Internet of Things. *Mob. Inf. Syst.* **2017**, *2017*, 1750637. [CrossRef]
21. Le, A.; Loo, J.; Chai, K.; Aiash, M. A Specification-Based IDS for Detecting Attacks on RPL-Based Network Topology. *Information* **2016**, *7*, 25. [CrossRef]
22. Fuller, A.; Fan, Z.; Day, C.; Barlow, C. Digital twin: Enabling technologies, challenges and open research. *IEEE Access* **2020**, *8*, 108952–108971. [CrossRef]
23. Eckhart, M.; Ekelhart, A. Towards security-aware virtual environments for digital twins. In Proceedings of the 4th ACM Workshop on Cyber-Physical System Security, Incheon, Republic of Korea, 4 June 2018; pp. 61–72.
24. Homaei, M.; Morales, V.G.; Gutierrez, O.M.; Gomez, R.M.; Caro, A. Smart Water Security with AI and Blockchain-Enhanced Digital Twins. *arXiv* **2025**, arXiv:2504.20275. [CrossRef]
25. El-Hajj, M. Leveraging Digital Twins and Intrusion Detection Systems for Enhanced Security in IoT-Based Smart City Infrastructures. *Electronics* **2024**, *13*, 3941. [CrossRef]
26. Schösser, A.; Khursheed, D.; Schulz, P.; Burmeister, F.; Fettweis, G. Digital Twin of the Radio Environment: A Novel Approach for Anomaly Detection in Wireless Networks. In *2023 IEEE Globecom Workshops (GC Wkshps)*; IEEE: Piscataway, NJ, USA, 2023. [CrossRef]
27. Bringye, Z. LoRa_FSM: Finite State Machine–Based Digital Twin for LoRaWAN OTAA. 2025. Available online: https://github.com/zsoltbringye/LoRa_FSM (accessed on 26 May 2025).
28. ChirpStack. ChirpStack Open-Source LoRaWAN Network Server. 2025. Available online: https://www.chirpstack.io/ (accessed on 26 May 2025).
29. Povalac, A.; Kral, J.; Arthaber, H.; Kolar, O.; Novak, M. Exploring lorawan traffic: In-depth analysis of iot network communications. *Sensors* **2023**, *23*, 7333. [CrossRef] [PubMed]
30. Elastic. Elastic Stack (ELK): Elasticsearch, Kibana & Logstash. 2025. Available online: https://www.elastic.co/elastic-stack (accessed on 26 May 2025).